Can Decision Models Replace Tool Calling for Coding Agents?

Aziz Muminov Alexandra Ilina

Preprint draft — 5 November 2026

Abstract

A coding agent connected to many MCP servers must either load every tool definition into its context (about 180k tokens for the 613 tools of twelve popular servers) or find tools through search, and harnesses ship different mechanisms: name lists with keyword search (Claude Code), BM25 (Codex, Anthropic's tool search, Hermes Agent), tool groups (GitHub Copilot), and embeddings or an LLM router (Goose). We ask whether a *decision model* (DM), here Microsoft Decision 1 (`microsoft/microsoft-decision-1` on OpenRouter's Decisions API), which returns calibrated probabilities over up to 255 options without generating text, is a better tool selector. We built a test suite from the real tool definitions of 30 MCP servers (613 and 1,076 tools) with simulated back ends, 71 coding tasks graded by hidden tests and required writes, and two agent harnesses (opencode and Hermes Agent). Our main experiment swaps the ranker inside Hermes Agent's own tool-search bridge (BM25, two DM variants, embeddings, a Claude Code-style ranker, random) on 38 symptom-first tasks over 1,076 tools with a pinned provider and a paired design (684 runs). Every ranker, even random, passed 96–100% of runs, but tokens differed: random used 1.57× the tokens of BM25, and a DM with a calibrated "none" option (DM v3) was the only ranker significantly below BM25, at 0.88× [95% CI 0.78, 0.98], about 12% fewer. Offline, a server-then-tool DM hierarchy ranks best (hit@1 0.93 at 613 tools against 0.48–0.54 for name, BM25 and embedding search and 0.86 for an LLM router), and on the agents' own queries its results contained the needed tool more often than BM25's (0.29 against 0.24). A projection from the measured token mix suggests that the DM's per-search fee pays off for agent models that cost more than about $0.02 per run.

1 Introduction

Coding agents now reach far beyond the repository. Through the Model Context Protocol (MCP) [17] an agent can read CI logs, error trackers, tickets, chat, dashboards, clusters and databases. Each MCP server contributes tools, and each tool a definition (name, description, parameter schema) that the agent's LLM must see before it can call the tool. Twelve widely used servers (GitHub, Grafana, Sentry, Atlassian, Linear, Slack, Notion, Kubernetes, MongoDB, Postgres, Git, Playwright) expose 613 tools whose definitions take about 180k tokens. Loaded on every request, they fill most of a context window before the agent has read a line of code.

Harnesses have responded with *tool search*: the definitions stay out of the prompt and the agent finds tools on demand. The mechanisms differ (§2). Claude Code lists the names of deferred tools and loads them by exact name or keyword [3]; Anthropic's API and OpenAI's Codex CLI offer BM25 search [2, 20]; GitHub Copilot groups tools into "virtual tools" that the agent activates [14]; Goose has offered embedding-based and LLM-based tool routers (documented as a preview in 2025) [6]; Hermes Agent defers MCP tools behind a BM25 bridge [18]. Each is a ranking function over the catalog, and they have not been compared with each other on the same tools and tasks.

Decision models (DMs) are a natural candidate for this ranking. A DM reads a state and a question with named options and returns a probability for each option in one forward pass, without generating tokens; we use Microsoft Decision 1 (MSD1; `microsoft/microsoft-decision-1`) [15] through OpenRouter's Decisions API [22]. Choosing a tool from described options is exactly such a question. A DM is, however, limited to 255 options per question, fewer than a realistic catalog holds, and it cannot write the tool's arguments.

We therefore ask: *can a DM serve as the tool selector of a coding agent, and is it better than the mechanisms that harnesses ship?* To answer it we built a test suite on real tool definitions (§4): the actual `tools/list` output of 30 MCP servers (613 tools, and 1,076 with natural decoys such as GitLab beside GitHub), simulated back ends that answer from per-task world rules, 71 tasks in two versions graded by hidden tests, required writes and the absence of destructive calls, eight selection conditions, and two harnesses. We report, with the caveats of §6:

- **Main experiment (§5.1).** Six rankers in Hermes Agent's shipped search bridge (BM25 as shipped, two DM variants, embeddings, a Claude Code-style ranker, random), so that hits are packaged identically and only the ranking differs; 38 tasks, 1,076 tools, DeepSeek V4.1 Flash on a pinned endpoint, three repeats, paired by task and repeat (684 runs). Every ranker passed 96–100% of runs; random used 1.57× the tokens of BM25, and DM v3, which can answer "none", was the only ranker significantly below it (0.88×, 95% CI 0.78–0.98).
- **Offline, the DM is the most accurate ranker (§5.3).** On 120 hand-written queries the DM hierarchy put the right tool first for 93% of queries at 613 tools and 88% at 1,076, against 45–54% for name, BM25 and embedding search and 82–86% for an LLM router, at about a third of the router's cost and less than half its latency. Asking for the server first is what makes the DM usable beyond 255 options; on queries written by agents the advantage is smaller.
- **In live agents, every search mechanism is at ceiling (§5.4).** With opencode, all eight catalog conditions passed 96–100% of runs on 25 tool tasks with two LLMs, and searching cut the largest prompt from about 182k tokens to 12–53k and the cost per run by up to 9×. This is why the main experiment measures tokens with a paired design on harder tasks.

Contributions. (i) A test suite for tool selection in coding agents built on real MCP tool definitions, with simulated back ends, graded tasks and an isolated sandbox. (ii) A DM-based tool ranker that works beyond the 255-option limit by choosing the server first, and its integration into a shipped harness's search bridge. (iii) A comparison of the mechanisms that current harnesses ship and of the DM, offline and inside two agents. (iv) A controlled, paired protocol for measuring token differences between agent configurations.

2 How harnesses select tools

Table 1 summarizes the mechanisms we compare. All except native loading share one pattern: the agent sees a small, fixed set of tools (a search tool and a way to call or load what it finds) plus, in most designs, a short overview of the servers, and pays for definitions only when it retrieves them.

Hermes's bridge. Hermes Agent 0.19.0 defers MCP tools once their definitions exceed 10% of the context window, which 613 tools do. Its bridge has three tools: `tool_search` returns compact hits (name and one line), `tool_describe` returns one full definition, and found tools are then called

Table 1: Tool-selection mechanisms in current harnesses and the conditions that reproduce them.

Condition	Shipped in	What the agent sees and does
native	Cursor, Cline, opencode; Claude Code with search off	every tool definition in every request
cc	Claude Code ToolSearch [3]	names of all deferred tools; `select:a,b` by exact name, or keyword search with name matches weighted highest (our approximation; the exact scoring is undocumented)
bm25	Anthropic tool search [2], Codex CLI [20], Hermes [18]	Okapi BM25 [24] over names, descriptions and parameters
groups	GitHub Copilot virtual tools [14]	one `activate_<group>` tool per server (servers over 40 tools split); activating loads the group's definitions
embed	Goose vector router [6], RAG-MCP [10]	cosine similarity of `text-embedding-3-small` vectors of query and tool
llm	Goose LLM router [6]	an LLM (DeepSeek V4.1 Flash) reads one line per tool and names the best ones
dm	this paper	the DM ranks servers, then tools (§3)
random	control	five random tools

directly with typed arguments. In our main experiment the DM takes BM25's place in this bridge (Figure 1), so the packaging is the same in every ranked condition.

Skills and progressive disclosure. Agent skills [1] apply the same idea to instructions: a one-line summary is always in context and the body is loaded on demand. The service overview that most search conditions include, and the "where things live" handbook of our v2 tasks (§4), play the role of such a summary for the catalog.

3 Decision models as tool selectors

The interface. A Decisions API request carries a *state* (here the agent's search query) and named questions; a `choice` question lists options, each with a free-text description, and returns a probability per option [22]. We describe each tool by its name, the start of its description (300 characters) and its parameter names. The DM in all our experiments is Microsoft Decision 1 (MSD1), called as `microsoft/microsoft-decision-1` on OpenRouter's Decisions API, a post-trained 9B model [15]; it answers in about half a second and costs about $0.04 per million input tokens.

The 255-option limit. A `choice` question accepts at most 255 options, and MSD1 rejects requests beyond about 32k tokens. We compared four ways around the limit:

- **Hierarchy** (default): ask which server has the needed tool (12 or 30 options, each described by the server's purpose), take the most likely servers until they cover 90% of the probability mass (at most three), ask which of their tools fits (Linear, the largest server, has 198), and score each tool by $P(\text{server}) \cdot P(\text{tool} \mid \text{server})$.
- **Tournament**: split the catalog into chunks of at most 250, keep the top five of each, and decide among the survivors.

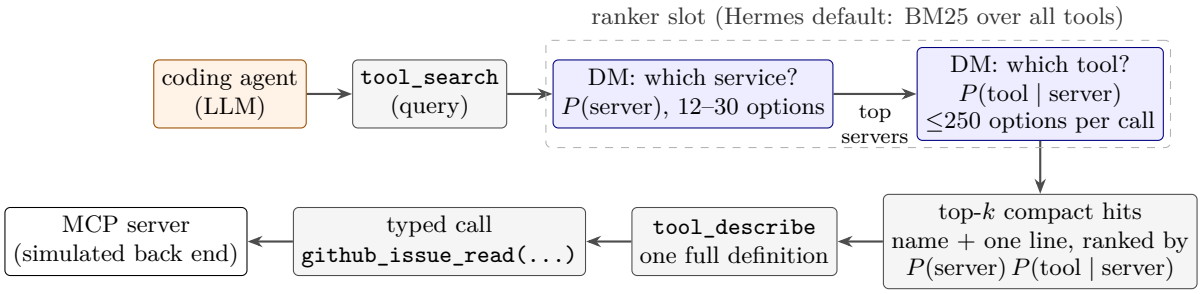


Figure 1: Tool selection with a decision model inside Hermes Agent's search bridge. The agent's query goes to the ranker slot, where the DM first chooses among servers and then among the tools of the most likely servers (at most 250 options per call). The agent receives compact hits, reads one full definition with `tool_describe`, and calls the tool directly with typed arguments. In the default harness BM25 fills the same slot; everything outside the dashed box is identical across conditions.

Table 2: The real tool catalog: tools per MCP server, from each server's own `tools/list`.

Catalog	Servers (tools)	Tools
main	Linear (198), Atlassian (98), GitHub (91), Grafana (81), Git (28), MongoDB (27), Playwright (25), Notion (24), Kubernetes (23), Sentry (9), Slack (8), Postgres (1)	613
extra	GitLab (127), monday (84), Salesforce (74), Heroku (33), HubSpot (21), AWS CloudWatch (19), AWS EKS (16), Airtable (16), Filesystem (14), CircleCI (13), Browserbase (9), Memory (9), Netlify (9), Terraform (9), Elasticsearch (4), ClickHouse (3), Figma (2), Docker (1)	+463
all	30 servers	1,076

- **BM25→DM** and **embed→DM**: shortlist 100 tools with BM25 or embeddings, then let the DM choose among them.

Figure 1 shows the hierarchy as it runs inside Hermes's bridge.

Why the DM selects and does not call. A DM chooses among options; it cannot produce the arguments of a call, and in a coding session most tool choices are easy. We checked this on 395 tool decisions of one real coding-agent session (the session in which this project was built). Given the user's request and the last eight steps, MSD1 predicted the agent's next tool among 38 with accuracy 0.49 (TypeSafe Jev [26]: 0.60), below the baseline "always the shell" at 0.75, although the DMs' top five contained the agent's tool 94–96% of the time. In the same session 51 always-loaded tool definitions took about 59k tokens, 93% of the first request and 16% of all context tokens the session read. The DM's place is therefore the catalog: deciding which of hundreds of rarely used tools to show, while the LLM keeps the core tools and writes every call.

4 Test suite

Real tool catalog. We started twelve MCP servers as an unprivileged user with dummy credentials and saved their `tools/list` responses: names, descriptions and input schemas exactly as the servers publish them (Table 2). No tool was ever executed against a real service. The resulting 613-tool catalog is our main setting. Eighteen further servers add 463 tools (1,076 in all). They include natural decoys for the main servers: GitLab beside GitHub, AWS CloudWatch beside Grafana Loki, AWS EKS beside Kubernetes.

Simulated back ends. The definitions are real; the answers are simulated. Each task has a world: a list of rules (tool, argument values to match, response), followed by organization-wide facts shared by all tasks (data sources, channels, teams). The first matching rule answers. Unmatched reads return what a real service returns for something that does not exist (an empty list, or not found); unmatched writes succeed and are recorded for grading. Optionally the simulator validates arguments against the tool's schema, as a real server would.

Tasks. Each task is a small Python repository whose fix needs information that lives only in the services: a failing CI check, a Sentry stack trace, Loki logs, a Prometheus alert threshold, a crash-looping pod, a Slack decision, a Jira or Linear ticket, a Confluence or Notion page, a MongoDB document shape, a Postgres schema. The sandbox has no network and no credentials, so the services are reachable only through the catalog. Many tasks also require a write: comment on the pull request, resolve the Sentry issue, move the ticket, reply in the thread, open a pull request.

- **v1** (33 tasks): 22 single-service, 3 multi-service, 4 *absent* (the information does not exist; the right outcome is to stop and leave a `TODO`), and 4 controls that need no catalog tool. Prompts often name the service ("check the logs in Loki").
- **v2** (38 tasks, Table 7): the same worlds with *symptom-first* prompts that do not name the service ("customers stopped getting shipping e-mails"), a shared "where things live" handbook in every README (like an `AGENTS.md`, naming the company's tools, including legacy GitLab, CloudWatch and EKS), and five new tasks, four of them multi-service and three on the extra servers, so that the decoy servers are sometimes the right answer.

Every prompt ends with the same escape clause: if the information is not available, do not guess; leave the code unchanged and add a `TODO: info not found` comment. In the final run (§5.1) every prompt also starts, identically in all conditions, with a note that stands in for what real harnesses say in the system prompt about connected MCP servers: "Note: the company's internal services (code hosting, CI, error tracking, metrics and logs, incidents and on-call, tickets, docs and runbooks, chat, clusters, databases) are connected to you as tools. Some tools are loaded on demand: if a tool you need is not listed, use tool search to find it."

Grading and validation. A run passes when (1) the hidden tests pass, (2) every required write was made with the right arguments, and (3) no destructive call was made (merge, delete, rollout restart, `kubectl apply`, ...). Before any agent saw a task we checked that its hidden tests fail on the original repository and pass with the reference solution, that every tool it names exists in the catalog, and that the reference calls get non-empty answers from the simulator (empty ones for absent tasks) and satisfy the required writes.

Harnesses. *opencode* [21] (v1.19.4) runs headless with a shell limited to `pytest` and file tools limited to the project. The catalog is served by our MCP server, which implements every condition of Table 1; the ranked-search conditions return the top five tools with full definitions and a service overview, and calls go through `call_tool(name, arguments)`. *Hermes Agent* [18] 0.19.0 uses its own bridge over our MCP server's full tool list. To compare rankers with identical packaging we patched the bridge's search function to ask our server's `/rank` endpoint, which runs the DM or a random ranker, when an environment variable is set (Figure 1).

Isolation. Every run executes as an unprivileged user with no network beyond the LLM proxy and the MCP server, and no credentials; the proxy holds the API key. Each Hermes run has its

own mount namespace (its own work directory, a fresh home directory and a private `/tmp`); the MCP server answers only requests that carry a per-run token, which Hermes removes from its shell's environment; and Hermes's `approvals.deny` list blocks commands that probe the machine (`curl`, `ps`, `netstat`, `/proc`, `localhost`, ...). App. A lists how we arrived at this setup.

5 Experiments

We first describe the main experiment and then the supporting experiments that show why a DM ranks tools well, how it gets past the 255-option limit, and why pass rate alone cannot separate rankers in a live agent.

5.1 Main experiment: rankers inside Hermes's search bridge

Design. The experiment holds everything fixed except the ranker in Hermes's bridge (Figure 1):

- Hermes Agent 0.19.0 with the isolation of §4 and its `approvals.deny` list;
- task set v2 (38 tasks) on the 1,076-tool catalog, with schema validation of arguments, and every prompt prefixed with the same note on connected services (§4);
- one LLM, DeepSeek V4.1 Flash [8], pinned to DeepInfra's fp8 endpoint, temperature 0.6, seed 7;
- six rankers in the same bridge: *Hermes BM25* (Hermes as shipped; Codex CLI also uses BM25); *DM v1* (service, then tool; top five); *DM v3* (the same with a calibrated "none" option: it returns nothing when $P(\text{none}) \geq 0.4$, and at most five tools otherwise); *embeddings* (`text-embedding-3-small`, as in Goose and RAG-MCP); *Claude Code style* (all deferred tool names in the prompt, keyword ranking and `select:<name>`); and *random* (five random tools, a lower bound);
- three repeats, $6 \times 38 \times 3 = 684$ runs. BM25, DM v1 and random were queued repeat by repeat with the conditions of each (task, repeat) slot back to back; DM v3, Claude Code style and embeddings were added later in parallel batches;
- analysis paired by slot: geometric mean of per-slot token ratios with a task-level bootstrap CI; pass@1 and pass^3 (the task passes in all three repeats).

Loading all definitions natively is not repeated here; at 613 tools it is covered by the opencode runs (§5.4). An oracle ranker is used only offline, to label the agents' searches (below).

Why paired, and why pinned. In pilot runs, the paired log ratio of tokens between two conditions had a standard deviation of about 0.44, so detecting a 10% token difference with 80% power at $\alpha = 0.05$ needs about 136 pairs; the design gives 114 per comparison. Unpinned, OpenRouter routed DeepSeek requests to dozens of endpoints with different quantizations, and the harness sets no temperature, which is why both are fixed here. We also piloted a second LLM, Kimi K3.1 Pro, and dropped it for cost and speed.

Grading. Two grading rules were corrected during the run: the destructive-call rule matched any tool name containing "merge", which also blocked a required merge-request comment, and one hidden test (`cw_reports`) rejected a valid fix. All runs were re-graded from their saved artefacts with the corrected rules (33 of 684 verdicts changed).

Table 3: Main experiment: Hermes Agent, DeepSeek V4.1 Flash (pinned), task set v2 (38 tasks), 1,076 tools, three repeats, 114 runs per ranker (684 in all, re-graded). pass^3: tasks passed in all three repeats. Tokens: median per run. Ratio: geometric mean of per-slot token ratios against BM25 with a task-level bootstrap 95% CI. Fewer: slots (of 114) on which the ranker used fewer tokens than BM25. Bold: best value per column (highest accuracy, fewest tokens); random's perfect pass rate is one run more than the others.

Ranker	pass@1	pass^3	Tokens	Ratio vs. BM25 [95% CI]	Fewer	Searches/run
BM25 (Hermes, Codex)	0.991	37/38	558k	–	–	11.4
DM v3 ("none" option)	0.991	37/38	**459k**	**0.88** [0.78, 0.98]	**69/114**	10.1
DM v1	0.991	37/38	480k	0.93 [0.82, 1.03]	66/114	9.7
embeddings	0.965	35/38	562k	0.91 [0.81, 1.03]	63/114	11.1
Claude Code style	0.991	37/38	576k	1.02 [0.89, 1.16]	43/114	6.0
random	**1.000**	**38/38**	1,051k	1.57 [1.39, 1.79]	27/114	12.8

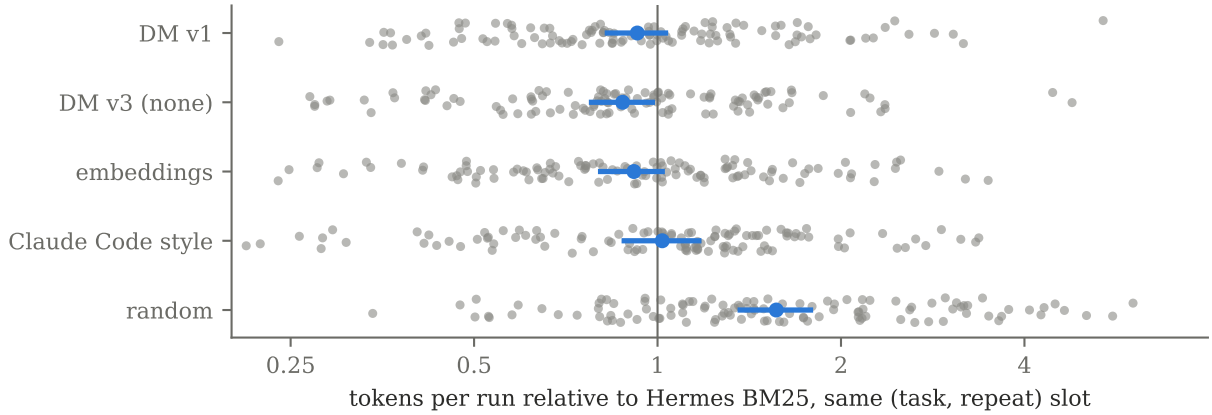


Figure 2: Main experiment: tokens per run relative to Hermes BM25 in the same (task, repeat) slot (114 slots per ranker). Grey: individual slots; blue: geometric mean with task-level bootstrap 95% CI.

Accuracy is at ceiling for every ranker, including random. All six rankers passed 96.5–100% of runs and abstained correctly on all 12 absent-task runs (Table 3). Even random ranking passed every run: an agent that gets five useless tools searches again until it finds what it needs. The remaining failures are real. In `k8s_crashloop` the agent ran a command in a production pod (`kubectl exec`), which our grading counts as destructive (once each under BM25, DM v1 and Claude Code style, twice under embeddings); in `cw_reports` the code fix was wrong (DM v3, embeddings); and once (embeddings, `prom_threshold`) the agent left the code unchanged.

Ranking shows up in tokens. Random ranking cost 1.57× the tokens of BM25 [1.39, 1.79] and used fewer tokens on only 27 of 114 slots, so the quality of the ranker matters. DM v3 was the only ranker significantly below BM25: 0.88× [0.78, 0.98], about 12% fewer tokens (median 459k against 558k per run), fewer on 69 of 114 slots, with fewer searches (10.1 against 11.4). DM v1 (0.93× [0.82, 1.03]) and embeddings (0.91× [0.81, 1.03]) leaned 7–9% lower, but their intervals include 1, and embeddings had the lowest pass rate. The Claude Code-style ranker searched least (6.0 per run), because the deferred names in the prompt let the agent select tools by name, but tied with BM25 in tokens (1.02×): listing 1,076 tool names costs context on every request.

Search accuracy in the live runs. To see whether the DM's ranking advantage survives the agents' own queries, we labelled every search in the BM25, DM v1, DM v3 and embeddings runs (4,785 searches from 408 runs) with the needed tool the agent was working towards. 2,862 searches were made while a needed tool was still missing, and 879 on tasks that need no tool. We replayed the 2,862 queries through each ranker (a hit: the five results contain a tool for the agent's current step). On identical queries the DM was right more often: DM v1 0.29, DM v3 0.28, BM25 0.24 and embeddings 0.24, and the order held whichever ranker's agent had written the queries (BM25 0.21–0.25, DM v1 0.27–0.30). On 228 queries DM v1 was right and BM25 wrong, against 84 the other way round (about 2.7:1). The results the agents actually received show the same order (BM25 0.23, embeddings 0.25, DM v1 0.27, DM v3 0.28). Absolute rates are low because many searches are exploratory or helper searches ("list all available services", a guess at a service the company does not use) from which no ranker can find the needed tool; every ranker was wrong on 1,890 queries. DM v3 answered "none" on 5.6% of its live searches (64 of 1,147); the ones we sampled asked for services that are indeed not in the catalog (AWS ECS, Google Drive, a Datadog-like APM). In the replay it answered "none" on 4.1% of the queries for which a needed tool did exist (117 of 2,862), and on 9% of searches from tasks that need no tool, where agents still search for plausible things such as "find the Jira ticket".

Pilot. An earlier pilot on 12 v1 tasks with two repeats and 613 tools (24 paired slots) pointed the same way: DM v1 used 0.88× the tokens of BM25 (95% CI 0.77–1.02).

5.2 Projected savings for frontier models

This subsection is a projection, not a measurement. The main experiment uses an inexpensive LLM, for which a 10% token saving is worth little. To estimate what the DM would save with more expensive agent models, we take the per-run token mix measured in the main run on the 71 (task, repeat) slots that both Hermes BM25 and the DM had finished (interim; means per run): BM25 used 34k uncached input, 747k cached input and 11k output tokens (output includes reasoning), the DM 35k, 657k and 10k. The DM's own fee was $0.0020 per run (about ten searches at about $0.0002 each). We price both mixes at each model's OpenRouter list price on 5 November 2026 and add the DM fee to the DM column (Table 4).

Assumptions. A frontier agent would produce a similar token mix and get a similar token reduction from the DM. Both may differ: stronger models may search less, or write better queries for BM25. The projection uses the interim token mix and ratio (DM v1, 71 slots: 0.90× [0.76, 1.05]); in the final run DM v3's ratio is 0.88× [0.78, 0.98] (Table 3), so a reduction of this size is supported for one model, but not yet for frontier models.

The fee is fixed; the saving scales with the agent's price. The DM's fee depends on the number of searches, not on the agent model, while the tokens it saves are billed at the agent's price. Under the assumptions above, break-even lies at about $0.02 of agent-LLM cost per run. Below it, as with the inexpensive model we measured, the DM makes a run about 3–7% more expensive; with any mid-range or frontier model in the table it makes a run about 7–8% cheaper, $19–80 per thousand runs for the models shown. The fee could be lowered further, for example by returning exact tool names found in the query without a DM call, caching repeated queries, or answering from the server step alone when the DM is confident.

Table 4: *Projected* LLM cost per run with Hermes BM25 and with the DM in the bridge (including the DM fee of $0.0020 per run), if each model used the token mix measured with DeepSeek V4.1 Flash in the main run (interim, 71 slots). Prices: OpenRouter list prices on 5 November 2026, $ per million tokens (input / cached input / output). Saving per 1,000 runs is BM25 minus DM; negative means the DM costs more. [a]Gemini 4 Argon has the same list prices as Claude Sonnet 5.5 and GPT-6.1 Sol, and hence the same projection.

Agent model	Price (in / cached / out)	BM25 $/run	DM $/run	Change	Saving/1k runs
DeepSeek V4.1 Flash (measured)	0.15 / 0.003 / 0.60	0.0142	0.0152	+7%	−$1.0
Claude Haiku 5.5	0.10 / 0.01 / 0.50	0.0166	0.0171	+3%	−$0.5
Claude Sonnet 5.5[a]	2.00 / 0.10 / 10.00	0.257	0.238	−7%	$19
Kimi K3.1 Pro	2.00 / 0.20 / 10.00	0.332	0.304	−8%	$28
GPT-6.1 Sol[a]	2.00 / 0.10 / 10.00	0.257	0.238	−7%	$19
Claude Fable 5.1	10.00 / 0.25 / 50.00	1.099	1.020	−7%	$80

Table 5: Offline retrieval on 120 hand-written queries. Hit@k: a right tool is among the top k. Cost per 1,000 queries and median latency of the selecting call at 613 tools; name, BM25 and embedding search run locally (embedding cost is negligible). Bold: best value per accuracy column.

	613 tools			1,076 tools			
Mechanism	hit@1	hit@5	MRR	hit@1	hit@5	$/1k queries	latency (s)
Claude Code-style (cc)	0.48	0.71	0.56	0.45	0.65	0	0.01
BM25	0.54	0.75	0.62	0.49	0.76	0	<0.01
embeddings	0.51	0.80	0.61	0.47	0.75	≈0	0.67
LLM router	0.86	**0.98**	0.92	0.82	**0.98**	0.44	1.31
DM hierarchy	**0.93**	0.97	**0.95**	**0.88**	0.97	0.15	0.54
DM tournament	0.90	**0.98**	0.94	0.86	**0.98**	1.13	1.60
BM25→DM	0.87	**0.98**	0.91	0.80	0.96	0.18	0.33
embed→DM	0.88	**0.98**	0.93	0.80	**0.98**	0.18	0.99

5.3 Why the DM ranks well: offline retrieval

We wrote two queries for each needed step of the v1 tasks, one in natural language and one as keywords (120 in all), and counted a hit when the top k contained any acceptable tool for that step. Table 5 and Figure 3 give the results.

The DM ranks best; the lexical and embedding searches that harnesses ship rank worst. At 613 tools the DM hierarchy put a right tool first for 93% of queries; the mechanisms of Claude Code, Codex and Goose's vector router managed 48–54%, and the LLM router 86%. All learned rankers reached hit@5 of 0.97–0.98, against 0.71–0.80 for the others. The DM hierarchy cost $0.15 per thousand queries, a third of the LLM router, at less than half its latency.

Choosing the server first is the right way past 255 options. Of the four strategies, the hierarchy was the most accurate at hit@1 and the cheapest DM variant. The tournament reached the same hit@5 but cost seven times as much, because it asks the DM about every chunk of the catalog; shortlisting with BM25 or embeddings first was as cheap but lost 5–6 points at hit@1, because the shortlist sometimes drops the right tool. Asking for the server first also matches how the tasks are structured: most needs lie within one server.

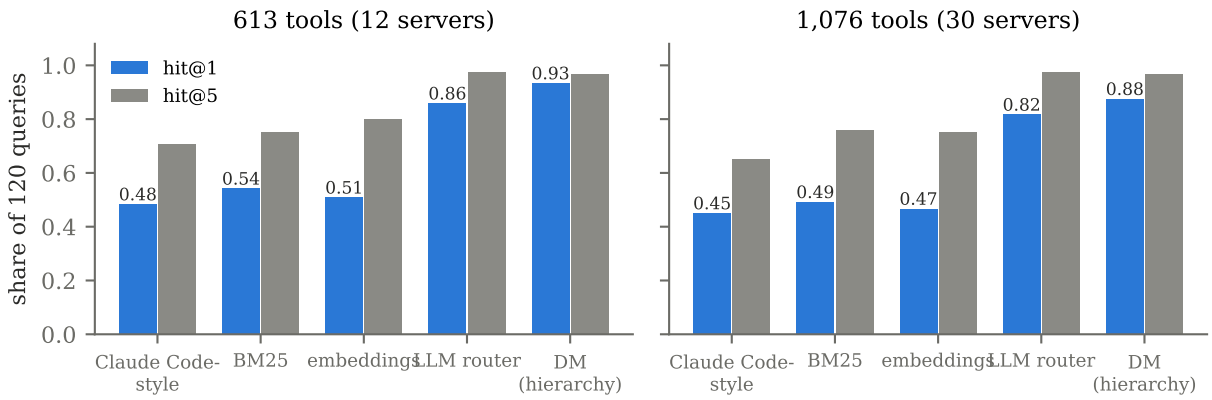


Figure 3: Offline retrieval, hit@1 (labelled) and hit@5, on the main and the extended catalog.

Table 6: opencode on task set v1, 613 tools (two repeats per task). Pass: runs passed on the 25 tasks that need a catalog tool. Largest prompt: median over those runs. Cost: mean LLM cost per run over all 33 tasks, excluding the selector (LLM router: \$0.0011/\$0.0031 per run; DM: \$0.0003/\$0.0010). Bold: best pass rate and smallest largest prompt among conditions with catalog access (*none* has no catalog and passes no tool task).

	DeepSeek V4.1 Flash			Tencent hy4		
Condition	Pass	Largest prompt	\$/run	Pass	Largest prompt	\$/run
none	0/25	9.6k	0.0110	0/25	15.4k	0.0154
native	**25/25**	181.9k	0.0722	**25/25**	188.5k	0.1039
cc	**25/25**	16.2k	0.0089	**25/25**	**32.0k**	0.0375
bm25	**25/25**	13.4k	0.0093	**25/25**	33.6k	0.0417
groups	**25/25**	29.0k	0.0179	24/25	52.7k	0.0562
embed	**25/25**	13.3k	0.0090	23/25	49.0k	0.0514
llm	**25/25**	**11.7k**	0.0076	**25/25**	32.6k	0.0419
dm	**25/25**	14.0k	0.0082	**25/25**	34.6k	0.0470

Decoy servers cost every mechanism a little. Adding 463 tools, including GitLab, CloudWatch and EKS beside GitHub, Loki and Kubernetes, lowered hit@1 by 3–6 points for every mechanism and left the order unchanged (DM 0.875, LLM router 0.82, BM25 0.49).

On queries written by agents the gap narrows. We replayed 183 search queries that agents wrote in our live runs, mostly lists of keywords and tool-name fragments. The DM still led at hit@1 (0.60 against 0.45 for BM25) but barely at hit@5 (0.70 against 0.66). Keyword-style queries suit lexical search, and agents compensate for a weak first query by searching again. Offline ranking quality therefore translates only partly into agent behaviour, which is why the main experiment measures the agent, not the ranker.

5.4 Eight mechanisms in a live agent: opencode

Each of the 33 v1 tasks ran under eight conditions with two LLMs, DeepSeek V4.1 Flash and Tencent hy4, twice for each pair of task and model (528 runs, 613-tool catalog). Table 6 and Figure 4 summarize them.

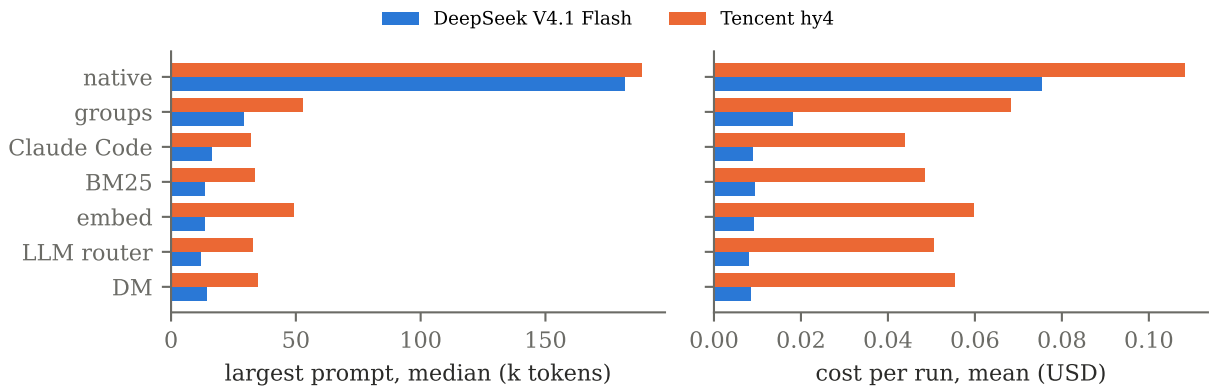


Figure 4: opencode, task set v1, 613 tools: median largest prompt on the 25 tool tasks and mean cost per run, by condition and LLM.

The tasks need the catalog, and every mechanism provides it. Without a catalog no tool task passed (0 of 50). With one, every condition passed 96–100%: native, cc, bm25, llm and dm 50 of 50, groups 49 and embed 48. All conditions abstained correctly on the four absent tasks (8 of 8 runs each), and no control run made a single catalog call. All three failures were hy4 runs: twice (groups, embed) the agent fixed the code but also restarted a production Kubernetes deployment it had not been asked to touch, which our grading counts as destructive, and once (embed) it stopped searching before it had found the ticket's specification.

Search removes most of the context. Loading every definition made the largest prompt about 182–189k tokens; with ranked search it was 12–16k (DeepSeek) and 32–35k (hy4), and 29–53k with groups, whose activation loads a whole server. That is 5–16× less, and the cost per run fell from \$0.072 to \$0.008–0.009 for DeepSeek and from \$0.104 to \$0.038–0.047 for hy4. Among the ranked searches the differences in tokens and cost are within run-to-run noise.

Runs use few tools. The median catalog run used 4 distinct catalog tools in about 10 calls, almost always from one server; 23 of the 25 v1 tool tasks need only one server. Selecting the right server is therefore most of the problem, which suits the DM hierarchy but also makes the task easy for any mechanism that shows a service overview. These ceiling results are why the main experiment uses task set v2 (symptom-first prompts, more multi-service tasks, decoy servers) and the larger catalog.

6 Threats to validity and lessons

Ceiling. On v1 every search mechanism passed nearly every run, so pass rate cannot rank them; we rank by tokens and cost, which are noisy. The v2 tasks (symptom-first prompts, more multi-service tasks, decoy servers) were meant to lower the ceiling; in the main experiment they did not (96–100% for every ranker, including random).

Simulated answers. Definitions are real, answers are not. Real services return longer, messier results, which would raise every condition's token count and could change how agents search. Unmatched reads return empty results, which is realistic but may make absent tasks easier to recognize than with a real service.

The connected-services note. Every prompt of the final run starts with the same note on connected services (§4), in all conditions including native. Real harnesses put a comparable statement in the system prompt, and our synthetic pilot showed that agents not told what exists may never search (App. B). The note names categories of services, not brands, so it does not tell the agent which server to search; it may still raise search rates above what an unprompted agent would show.

Approximated mechanisms. We reimplemented Claude Code's ToolSearch, Copilot's virtual tools and Goose's routers from their documentation rather than running those harnesses; only Hermes's bridge is the shipped code. Our cc scoring in particular is an approximation.

Query distribution and scale. Hand-written retrieval queries overstate the DM's advantage; agent-written queries are shorter and more lexical (§5.3). The live results so far rest on two inexpensive LLMs and one DM, and the main experiment uses a single LLM.

Packaging. The opencode conditions return full definitions and route calls through a generic `call_tool`, while Hermes returns compact hits and lets the agent call tools directly. Comparisons of rankers are therefore made within one harness; the main experiment holds the packaging fixed and varies only the ranker.

Power. With 114 pairs per comparison the main experiment is slightly short of the roughly 136 pairs estimated to detect a 10% token difference; differences of that size may remain inconclusive.

7 Related work

Tool retrieval for large catalogs. Retrieving tools before generation is the standard answer to large catalogs [7, 23]. RAG-MCP [10] retrieves MCP tool descriptions with embeddings to reduce prompt bloat, and MCP-Zero [9] lets the agent request tools actively by describing what it needs, server first and tool second, the same hierarchy our DM uses. Lee [12] find that selecting a tool is largely retrieval. We compare these ideas with the mechanisms harnesses actually ship, and with a DM, inside unmodified agents.

MCP benchmarks. LiveMCPBench [16] evaluates agents that must navigate a large set of real MCP servers, and MCPMark [27] stress-tests realistic MCP use with verifiable tasks; MCP-Atlas [5] and Toolathlon [13] also score task completion with real tools. They measure agents' end-to-end success; we hold the agent fixed and vary only the selection mechanism, with simulated back ends so that runs are reproducible and writes can be graded safely.

Vendor reports. Stacklok compared its MCP Optimizer with Anthropic's tool search tool on 2,792 tools and reported much higher selection accuracy for its hybrid retrieval [25]; Arcade tested Anthropic's tool search on about 4,000 tools [11]. Both measure selection accuracy rather than agents completing tasks, and neither reports token costs per task.

Decision models. The Artificial Analysis Decision Index [4] and an analysis of it [19] compare DMs with LLMs on standalone decisions, including tool routing. We place a DM inside agents, as one component of tool search.

8 Conclusion

A decision model can serve as a coding agent's tool selector. On real MCP tool definitions it ranks tools more accurately than the lexical and embedding searches that current harnesses ship, and more cheaply than an LLM router, and asking for the server first takes it past its 255-option limit. Whether this makes agents better is a different question: in live runs every search mechanism, the DM included, reaches ceiling pass rates and removes most of the context that loading all tools costs, and agents' own queries narrow the DM's ranking advantage. In our main experiment, with rankers swapped inside Hermes Agent's shipped bridge on harder tasks and a larger catalog with decoys, every ranker still reached ceiling pass rates; random ranking cost 57% more tokens than BM25, and the DM with a calibrated "none" option was the only ranker significantly cheaper, at about 12% fewer tokens (0.88×, 95% CI 0.78–0.98). On the agents' own queries the DM found the needed tool more often than BM25. If a similar reduction holds for other agent models, a projection from the measured token mix puts the DM at 7–8% cheaper per run for mid-range and frontier agent models, and slightly more expensive for the inexpensive model we measured (§5.2).

Reproducibility. The tool harvester, the catalog snapshots, the simulator, the 71 tasks with hidden tests and reference solutions, the MCP server with all selection conditions, the Hermes patch, the sandbox setup, all run records and the analysis and figure scripts will be released.

Use of AI assistance. Claude Code, Anthropic's coding agent, helped throughout this work: it wrote the harness, catalog harvester, tasks and analysis scripts, ran the experiments, and drafted the paper, under the authors' direction. The authors chose the research questions, reviewed the code, results and text, and are responsible for the content. The session analysed in §3 is this assistant's own session. [TODO: authors: confirm or revise the AI-use statement.]

References

[1] Anthropic. Agent skills overview (Claude platform documentation). https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview, 2025. Accessed 5 November 2026.

[2] Anthropic. Tool search tool (Claude platform documentation; regex and BM25 variants). https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool, 2025. Accessed 5 November 2026.

[3] Anthropic. Claude code documentation: Scale with MCP tool search. https://code.claude.com/docs/en/mcp, 2026. Accessed 5 November 2026.

[4] Artificial Analysis. Artificial analysis decision index v1.0. https://artificialanalysis.ai/evaluations/artificial-analysis-decision-index, 2026.

[5] Chaithanya Bandi et al. MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers. *arXiv preprint arXiv:2602.00933*, 2026.

[6] Block. goose: Tool router (vector-based) and LLM-based tool selection strategy. https://web.archive.org/web/20251031035432/https://block.github.io/goose/docs/guides/managing-tools/tool-router/, 2025. Archived 31 October 2025 (preview feature); not listed in the current documentation at goose-docs.ai. Source: https://github.com/block/goose.

[7] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference. *arXiv preprint arXiv:2603.13426*, 2026.

[8] DeepSeek. DeepSeek V4.1 Flash. Model card, via OpenRouter (`deepseek/deepseek-v4.1-flash`), 2026.

[9] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero: Active tool discovery for autonomous LLM agents. *arXiv preprint arXiv:2506.01056*, 2025.

[10] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[11] Eric Gustin. Anthropic tool search test: 4,000 tools, 60% success. https://www.arcade.dev/blog/anthropic-tool-search-4000-tools-test/, December 2025. Arcade.dev blog, 3 December 2025.

[12] Janghoon Lee. Selection is retrieval, abstention is not: On-device tool routing over 70 Korean-English actions. *arXiv preprint arXiv:2609.18672*, 2026.

[13] Junlong Li et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.

[14] Microsoft. Use tools with agents (Visual Studio Code documentation: 128-tool limit and virtual tools). https://code.visualstudio.com/docs/copilot/agents/agent-tools, 2026. Accessed 5 November 2026; see also the GitHub blog post "How we're making GitHub Copilot smarter with fewer tools".

[15] Microsoft. Not every decision needs an LLM: Getting started with Microsoft-Decision-1. https://techcommunity.microsoft.com/blog/educatordeveloperblog/not-every-decision-needs-an-llm-getting-started-with-microsoft-decision-1/4563468, October 2026. Microsoft Community Hub, Educator Developer Blog.

[16] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Qianhao Yuan, Xuanang Chen, Yaojie Lu, et al. LiveMCPBench: Can agents navigate an ocean of MCP tools? *arXiv preprint arXiv:2508.01780*, 2025.

[17] Model Context Protocol. Model context protocol specification. https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[18] Nous Research. Hermes agent. https://github.com/NousResearch/hermes-agent, version 0.19.0 (PyPI), 2026.

[19] Randal S. Olson. When to use a decision model instead of an LLM. Blog post, 15 October 2026, https://www.randalolson.com/2026/10/14/when-to-use-a-decision-model-instead-of-an-llm/, 2026.

[20] OpenAI. Codex CLI: open-source coding agent; MCP tool search with BM25 in `codex-rs/core/src/tools/handlers/tool_search.rs`. https://github.com/openai/codex, 2026. Source checked 5 November 2026.

[21] opencode contributors. opencode: an open-source coding agent. https://opencode.ai, version 1.19.4, 2026.

[22] OpenRouter. Decisions API reference. https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request, 2026.

[23] Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.

[24] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[25] Stacklok. Stacklok's MCP optimizer vs Anthropic's tool search tool: A head-to-head comparison. https://stacklok.com/blog/stackloks-mcp-optimizer-vs-anthropics-tool-search-tool-a-head-to-head-comparison/, 2025.

[26] TypeSafe AI. Jev 1.14 decision model. https://typesafe.ai; served via OpenRouter as `typesafe/jev-1.14`, 2026.

[27] Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, et al. MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use. *arXiv preprint arXiv:2509.24002*, 2025.

A Harness hardening

The setup of §4 is the result of fixes made during development; runs made before them are not reported. In order:

- Agents run as an unprivileged user with an empty environment and no credentials; experiment files are readable only by root and the API key is held by a local proxy that admits only the experiment models (harness-level permissions alone let an agent read hidden tests through absolute paths).
- Each Hermes run gets its own mount namespace: a private work directory, home directory and `/tmp`, so runs cannot see each other's files or leftovers.
- The MCP server accepts only requests carrying a per-run token that the agent's shell cannot read, so the shell cannot list the catalog or query the ranker directly.
- Hermes's `approvals.deny` list blocks probing commands (`curl`, `wget`, `ps`, `ss`, `netstat`, `nmap`, `lsof`, `/proc`, `localhost`, `find /`).
- Rankers are compared inside Hermes's own bridge (compact hits, `tool_describe`, typed calls) rather than through a separate MCP-side search with a generic `call_tool`, so that only the ranking differs.
- The provider and sampling are pinned (DeepInfra fp8, temperature 0.6, seed 7) and the conditions of each (task, repeat) slot are queued back to back.

B The earlier synthetic pilot

An earlier version of this study used a fictional company's catalog of 100 and 300 synthetic tools (about 610 tokens each) with opencode, and compared loading all tools, a DM-backed `find_tool`/`call_tool` pair with and without a service list, and tools preselected from the task prompt.

It showed the same pattern as the real-catalog runs: search matched native loading at a fraction of the context, and an agent without a list of the services sometimes never searched. We replaced it because its templated descriptions were more uniform and easier to rank than real ones, its tasks were at ceiling, and its tools resembled no real server, so it could not compare the mechanisms that harnesses ship. Its code and results remain in the repository.

C Task set v2

Table 7: Task set v2 (38 tasks). Prompts are shortened to their first sentence and shown without the escape clause. Services: the servers of the needed tools, in order; $+w$: number of required writes; $\star$: new in v2.

Category	Task	Prompt (first sentence)	Services
control	`c_chunk`	chunk in batching/chunk.py drops the last partial chunk.	–
control	`c_dedupe`	dedupe in collections_util/dedupe.py loses the original order.	–
control	`c_duration`	parse_duration in timeutil/duration.py only understands hours.	–
control	`c_leap`	is_leap in calendar_util/leap.py is wrong for some years.	–
single	`confluence_limits`	Our public API rate limits changed.	atlassian
single	`eks_importer` $\star$	The data team's nightly batch-importer job has failed three nights in a row.	aws_eks
single	`gh_ci_fail`	Pull request #214 in acme/payments can't be merged because a check is red.	github $+1w$
single	`gh_codesearch`	SUPPORTED_CURRENCIES in shop/currencies.py has drifted from the company-wide canonical list, which lives in another repository.	github
single	`gh_dependabot`	Security says acme/payments has dependencies with known vulnerabilities.	github
single	`gh_issue`	Please implement acme/payments#301.	github
single	`gh_open_pr`	Pagination in acme/catalog is broken (see #57).	github $+2w$
single	`gh_release`	We are behind on the ledger API.	github
single	`gh_review`	There is reviewer feedback waiting on pull request #88 in acme/ledger.	github $+1w$
single	`grafana_incident`	We are in the middle of an incident about checkout timeouts and the responders agreed on a mitigation.	grafana $+1w$
single	`jira_impl`	Pick up PAY-412: implement it, then move it to review.	atlassian $+1w$
single	`k8s_crashloop`	ledger-worker keeps restarting in production (namespace ledger) since the last config change.	kubernetes
single	`k8s_oom`	report-builder (namespace reports) keeps dying in production.	kubernetes
single	`linear_bug`	Fix ENG-77 and move it to In Review.	linear $+1w$
single	`loki_errors`	Customers stopped getting shipping e-mails at around 06:00 UTC today; the notifier service is the suspect.	grafana
single	`mongo_shape`	The nightly export has started crashing on recent orders.	mongodb
single	`notion_runbook`	We are rotating the webhook signing keys.	notion
single	`oncall_contact`	Set ESCALATION_CONTACT in ops/escalation.py to the e-mail address of whoever is on call for the payments team right now.	grafana

Category	Task	Prompt (first sentence)	Services
single	`pg_migration`	Add migrations/0042_payments_merchant_idx.sql with an index named payments_merchant_created_idx for looking up a merchant's payments by creation time.	postgres
single	`prom_threshold`	Set SLOW_REQUEST_MS in alerts/config.py to twice the checkout service's current p99 request latency, in milliseconds.	grafana
single	`sentry_events`	Since this morning's deploy some clients get HTTP 500 from POST /refunds.	sentry
single	`sentry_fix`	The noisiest production error of payments-api right now needs to go.	sentry +1w
single	`slack_decision`	The payments team agreed on a new webhook retry policy in a chat discussion.	slack +1w
multi	`cw_reports` ⋆	The legacy reports API has been answering 502 since yesterday evening.	aws_cloudwatch, slack +1w
multi	`gl_pipeline` ⋆	The main-branch pipeline of billing-legacy is red.	gitlab +1w
multi	`linear_notion_pr` ⋆	Do ENG-91 the way our API guidelines say, and open a pull request for it from a branch named eng-91 against main in acme/public-api.	linear, notion, github +1w
multi	`multi_alert`	Something is paging for the checkout service.	grafana, slack +1w
multi	`multi_jira_conf`	Implement PAY-530 as its design document specifies, then leave a one-line summary on the ticket.	atlassian +1w
multi	`multi_sentry_gh`	A recent change to ledger-worker introduced a reconciliation bug that shows up as a production error.	sentry, github +1w
multi	`promo_sentry_jira` ⋆	Android users report they can't apply promo codes.	sentry, atlassian +1w
absent	`absent_jira`	Implement PAY-999 in payments/limits.py.	–
absent	`absent_release`	Pin SDK_VERSION in sdk/version.py to the newest release of acme/ghost-sdk (without the leading v).	–
absent	`absent_sentry`	The billing-cron error someone mentioned in standup needs fixing in billing/cron.py.	–
absent	`absent_slack`	Set CACHE_TTL_S in cache/config.py to the value the platform team agreed on.	–